

CSE420 Compiler Design

Topic: Runtime Storage Organization: Stack

Prepared by Azmari Sultana

Table of Contents

1. Introduction to Runtime Storage Organization	3
1.1 Why a Stack?	3
2. Activation Trees	3
2.1 Procedures and Activations	3
2.2 Nesting of Activations	3
2.3 The Activation Tree	3
2.4 Worked Example – Quicksort Activation Tree	4
3. Activation Records	6
3.1 The Control Stack	6
3.2 Contents of a General Activation Record	6
3.3 Worked Example – Locating an Activation Record on the Stack	7
3.4 Downward Growth of the Stack	8
3.5 Complete Stack Simulation for the Quicksort Run	9
4. Calling Sequences	16
4.1 Calling and Return Sequences	16
4.2 Design Principles	16
4.3 The top_sp Register	17
4.4 The Calling Sequence, Step by Step	17
4.5 Variable-Length Data on the Stack	18

1. Introduction to Runtime Storage Organization

Every running program needs run-time memory for three broad kinds of things: the compiled code itself, static/global data, and the data that belongs to individual procedure calls. Compilers typically split this run-time memory into regions – commonly a **static area**, a **stack**, and a **heap**. Objects that are created and destroyed unpredictably, not tied to any single procedure call, are kept on the **heap**, where a garbage collector reclaims space that is no longer reachable. This note is about the other major region – the **run-time stack** – which is where almost every compiler places the data belonging to procedure, function, or method calls.

1.1 Why a Stack?

Almost all compilers for languages built around procedures, functions, or methods manage at least part of their run-time memory as a stack. Each time a procedure is called, space for its local variables is **pushed** onto the stack; when the procedure terminates, that space is **popped** back off. This works because of one key property of ordinary procedure calls: **activations nest in time**. If an activation of *p* calls an activation of *q*, then *q* must finish before *p* can finish – which is exactly the discipline a last-in-first-out stack enforces.

Stack allocation earns its keep in two ways:

- Memory used by calls whose durations do not overlap in time can be **shared and reused** – one call's space is freed and is immediately available to whichever call is pushed next.
- The compiler can generate code for a procedure so that the **relative addresses of its non-local data stay the same** every time it runs, no matter what sequence of calls led up to this particular invocation.

2. Activation Trees

2.1 Procedures and Activations

"Procedure" is used as a generic term for a function, procedure, method, or subroutine. Each time a procedure actually runs – from the moment it is entered to the moment it returns – that run is called an **activation** of the procedure. A recursive procedure may have several activations alive at the same time; each is a separate activation with its own parameters and locals.

2.2 Nesting of Activations

If an activation of *p* calls procedure *q*, that activation of *q* must end before the activation of *p* can end. There are exactly three ways this can happen:

1. **Normal termination** – *q* terminates normally, and (in essentially any language) control resumes in *p* right after the point of the call.
2. **Abort** – *q*, or some procedure that *q* calls (directly or indirectly), aborts, so it becomes impossible for execution to continue. In that case *p* ends at the same moment as *q*.
3. **Unhandled exception** – *q* terminates because of an exception it cannot handle. If *p* can handle the exception, the activation of *q* has ended while *p* continues (not necessarily from the point of the call). If *p* cannot handle it either, *p* terminates together with *q*, and presumably some other open activation further up will handle it.

2.3 The Activation Tree

Because activations always nest in time, we can represent every activation that occurs during an entire run of a program as a tree – the **activation tree**:

- Each **node** corresponds to exactly one activation.
- The **root** is the activation of `main`, which starts the whole program.
- At the node for an activation of `p`, the **children**, listed left to right, are the activations called by that activation of `p`, in the order they are called. One child must finish before the activation to its right can begin.

Three relationships tie this tree directly to the run-time behaviour of the program:

1. The sequence of procedure **calls** corresponds to a **preorder traversal** of the activation tree.
2. The sequence of **returns** corresponds to a **postorder traversal** of the activation tree.
3. If control currently lies inside the activation at node `N`, the activations that are currently open (**live**) are exactly `N` and its **ancestors**. They were called in the order they appear along the path from the root down to `N`, and they will return in the **reverse** of that order.

2.4 Worked Example – Quicksort Activation Tree

A sketch of a program that reads nine integers into array `a` and sorts them with recursive quicksort:

```
int a[11];
void readArray() { /* Reads 9 integers into a[1..9] */
    int i;
    ...
}
int partition(int m, int n) {
    /* Picks a separator value v, and partitions a[m..n] so that
       a[m..p-1] < v, a[p] = v, and a[p+1..n] >= v. Returns p. */
    ...
}
void quicksort(int m, int n) {
    int i;
    if (n > m) {
        i = partition(m, n);
        quicksort(m, i-1);
        quicksort(i+1, n);
    }
}
main() {
    readArray();
    a[0] = -9999; /* low sentinel */
    a[10] = 9999; /* high sentinel */
    quicksort(1, 9);
}
```

`main` has three jobs: read the data, plant two sentinel values (so `partition` never runs off either end of the array), and sort the real data with a single call to `quicksort(1, 9)`.

One possible enter/leave sequence for a run of this program is shown below (functions abbreviated by their first letter: `m` = `main`, `r` = `readArray`, `p` = `partition`, `q` = `quicksort`). In this run, `partition(1, 9)` happens to return 4, so `a[1..3]` hold the smaller elements and `a[5..9]` hold the larger ones:

```

enter main()
  enter readArray()
  leave readArray()
  enter quicksort(1,9)
    enter partition(1,9)
    leave partition(1,9)
    enter quicksort(1,3)
    ...
    leave quicksort(1,3)
    enter quicksort(5,9)
    ...
    leave quicksort(5,9)
  leave quicksort(1,9)
leave main()

```

One activation tree that completes this sequence is shown below. (Remember: the exact shape depends on what partition returns each time – this is only one possibility.)

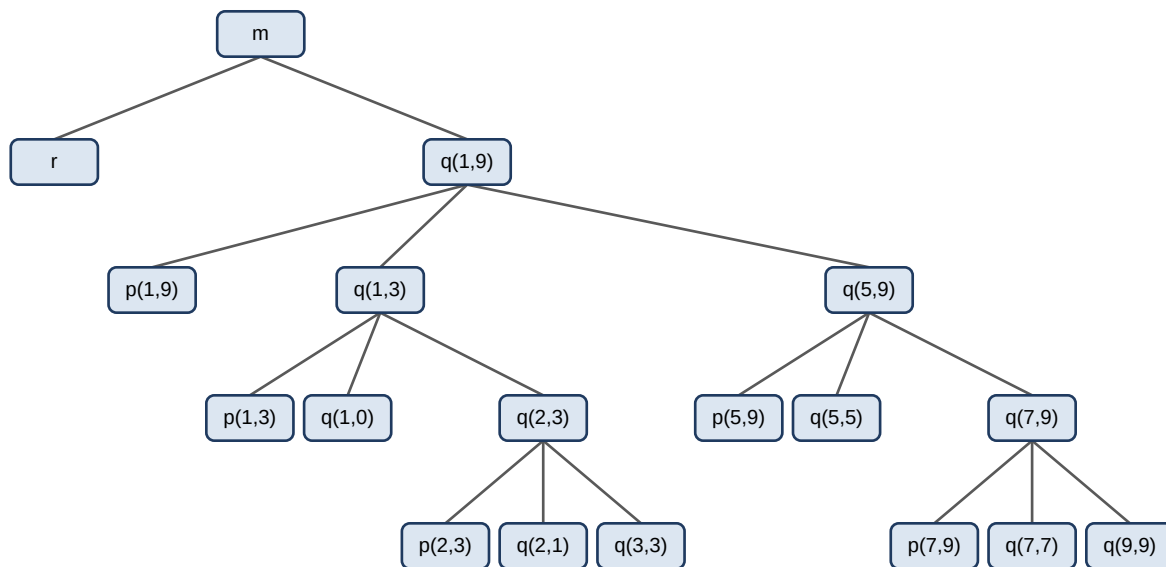


Figure 2.1 – Activation tree for the quicksort run above ($m = \text{main}$, $r = \text{readArray}$, $p = \text{partition}$, $q = \text{quicksort}$).

Check – the returned pivot index at each partition call is fully determined by its two children in the tree: $\text{partition}(1,9)=4$, $\text{partition}(1,3)=1$, $\text{partition}(2,3)=2$, $\text{partition}(5,9)=6$, $\text{partition}(7,9)=8$. For example, $\text{quicksort}(1,3)$ calls $\text{quicksort}(1, i-1)$ and $\text{quicksort}(i+1, 3)$; its children are $q(1,0)$ and $q(2,3)$, so $i-1=0$ and $i+1=2$, giving $i=1$ – consistent both ways.

Reading the tree confirms all three relationships from Section 2.3. The enter-sequence above is a preorder walk of Figure 2.1; the leave-sequence is a postorder walk; and, for example, when control is inside $q(2,3)$, the live activations are exactly $q(2,3)$, $q(1,3)$, $q(1,9)$, and m – the path from the root to $q(2,3)$.

Theory Questions

Q. What is an activation of a procedure, and what does the root of an activation tree represent?

A. An activation is one execution of a procedure – one call, from entry to return. The root of the activation tree is the activation of `main`, the procedure that starts the whole program.

Q. What property of procedure calls makes stack allocation feasible at all?

A. Procedure activations must nest in time: if `p` calls `q`, the activation of `q` must end before the activation of `p` ends. Because of this strict last-in-first-out nesting, a stack can always hold exactly the set of currently live activations.

Q. State the three relationships between an activation tree and the run-time behaviour of the program.

A. (1) The sequence of calls is a preorder traversal of the tree. (2) The sequence of returns is a postorder traversal of the tree. (3) If control is at node `N`, the live activations are exactly `N` and its ancestors, called in root-to-`N` order and returning in the reverse order.

Q. In what three ways can an activation of `q`, called by `p`, come to an end?

A. (1) Normal termination – control resumes in `p` right after the call. (2) `q`, or something it calls, aborts, so `p` ends at the same time as `q`. (3) `q` terminates via an exception it cannot handle – `p` may catch it and continue, or, if `p` cannot handle it either, `p` terminates together with `q`.

Q. At the moment `partition(2,3)` is called, which activation records are simultaneously live on the stack?

A. `p(2,3)`, `q(2,3)`, `q(1,3)`, `q(1,9)`, and `main` – reading bottom-up, this is exactly the path from the root to `p(2,3)` in the activation tree of Figure 2.1.

Q. After `q(2,3)` returns, whose activation record becomes the new top of the stack, and why?

A. `q(1,3)`'s – because `q(1,3)` is `q(2,3)`'s parent (caller) in the activation tree, and the caller's record is always immediately below the record of whichever activation it most recently called.

3. Activation Records

3.1 The Control Stack

Procedure calls and returns are usually managed by a run-time stack called the **control stack**. Every live activation has an **activation record** (sometimes called a **frame**) on the control stack. The record for the root of the activation tree sits at the bottom; the entire sequence of records on the stack, from bottom to top, corresponds exactly to the path in the activation tree from the root down to the activation where control currently resides. The record for that current activation sits at the **top** of the stack.

*Diagram convention used in this note – every stack diagram below is drawn with the current top of the stack at the **top** of the picture, and the bottom of the stack (globals, then `main`) at the bottom, growing downward as more calls nest. Your textbook draws the same information upside-down ("bottom of the stack higher than the top," so the elements lowest on the printed page are closest to the top of the stack) – if you ever compare directly against a textbook figure, just flip the picture top-to-bottom; the field order inside each record is unaffected.*

3.2 Contents of a General Activation Record

The exact contents vary with the language being implemented, but an activation record typically carries some subset of the following fields. They are listed below from the top of the stack (most recently added, part of the call currently executing) down to the bottom of this frame (closest to the caller):

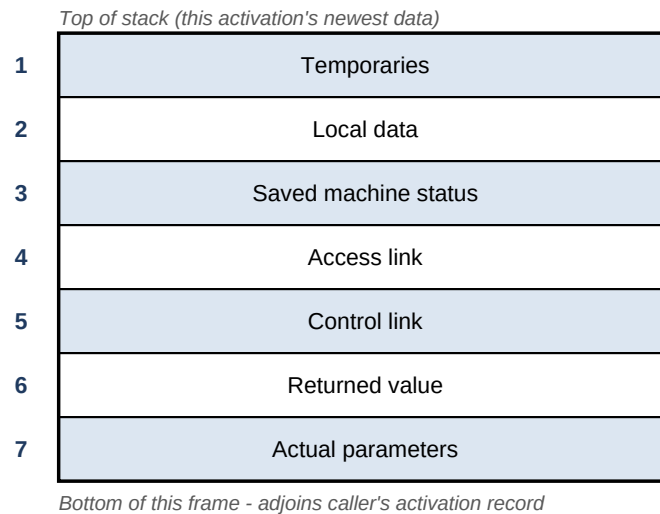


Figure 3.1 – A general activation record.

1. **Temporaries** – values from evaluating expressions, for the cases where a temporary cannot be kept in a register.
2. **Local data** – the local variables that belong to the procedure owning this record.
3. **Saved machine status** – the state of the machine just before this call: the **return address** (the program-counter value control must go back to) and the contents of any registers the caller had in use that must be restored when the call returns.
4. **Access link** – (optional) used to locate data the callee needs but that actually lives in another activation record elsewhere on the stack; used for nested-scope / non-local access.
5. **Control link** – a pointer to the caller's activation record.
6. **Returned value** – space for the value the called function sends back, if any (often kept in a register instead, for speed).
7. **Actual parameters** – the arguments supplied by the caller (often kept in registers when there are few enough; shown here for generality).

3.3 Worked Example – Locating an Activation Record on the Stack

Suppose control currently lies inside q(2,3) of the quicksort activation tree (Figure 2.1). By the live-activation rule, the stack – top to bottom – holds exactly: q(2,3) at the top, then q(1,3) (its parent), then q(1,9), and finally m (main) at the bottom.

Top of stack

integer i
q(2,3)
integer m, n
integer i
q(1,3)
integer m, n
integer i
q(1,9)
integer m, n
main
integer a[11] (global)

Figure 3.2 – Control stack while control is inside q(2,3) – compare against step 12 in Figure 3.4 (the same stack shape recurs at steps 14, 16, and 18, whenever control is back in q(2,3) itself).

3.4 Downward Growth of the Stack

As control flows through the activation tree, the run-time stack grows and shrinks. Because array a is global, space for it is allocated (right at the bottom, below main) before execution even begins. Tracking the first few calls of the quicksort run:

- Only main's record sits above the global data.
- The first call in main's body activates readArray (r); its record – holding local variable i – is pushed on top.
- When r returns, its record is popped, leaving only main. Then quicksort (1, 9) is called; its record (holding parameters m, n and local i) is pushed, reusing the very space r had just used. No trace of r's data survives for q(1,9) to see.
- Several activations later – after a recursive call to q(1,3), and after p(1,3) and q(1,0) have each run to completion and been popped – the record for q(1,3) is back on top of the stack.

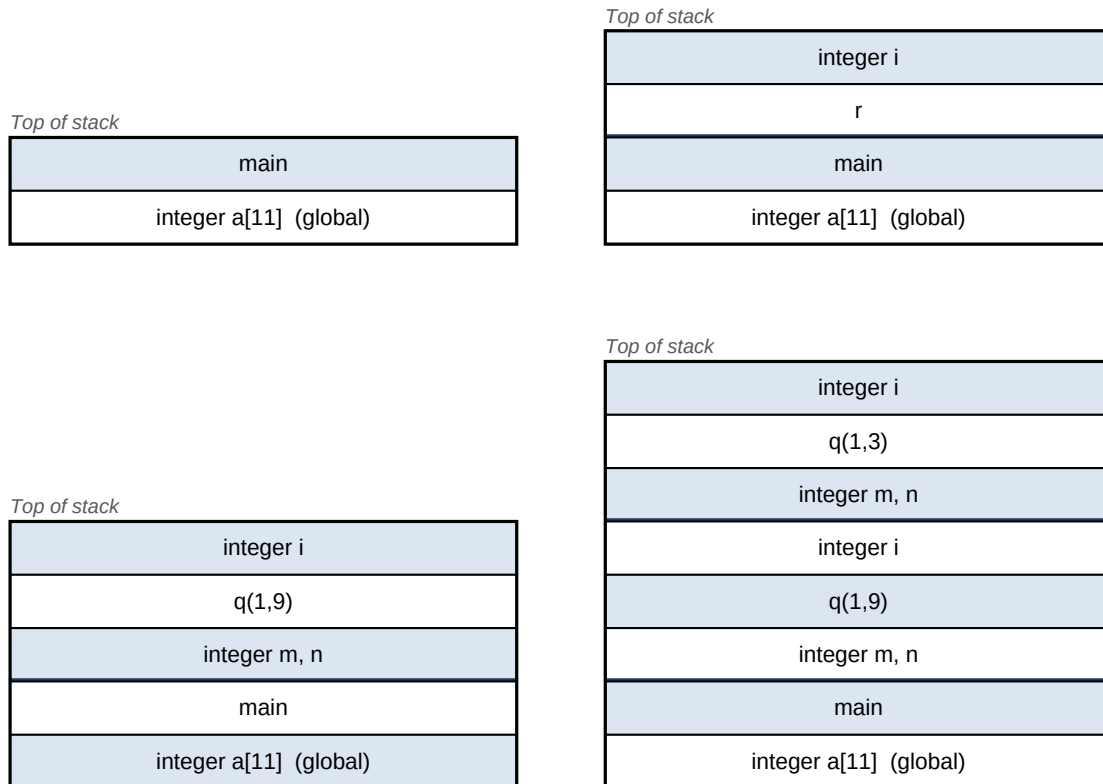
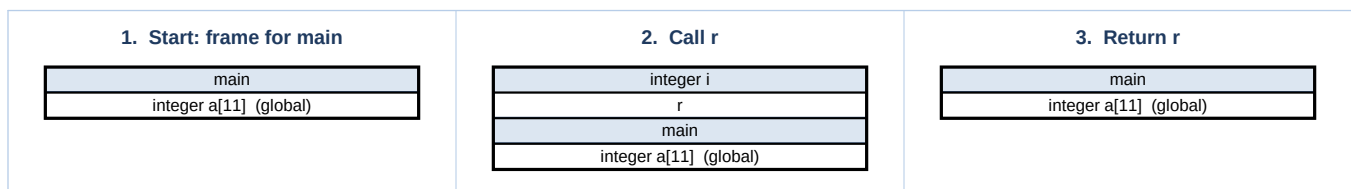


Figure 3.3 – Downward growth of the control stack: (a) frame for `main`; (b) `r` activated; (c) `r` popped and `q(1,9)` pushed; (d) control back at `q(1,3)`.

Because quicksort is recursive, it is completely normal to have several of its own activation records on the stack at once (e.g. `q(1,9)`, `q(1,3)`, and – deeper still – `q(2,3)` can all be live simultaneously; see the complete trace in Section 3.5.

3.5 Complete Stack Simulation for the Quicksort Run

We now have everything we need – the activation tree of Figure 2.1 (Section 2.4) fixes the order of calls and returns, and Section 3.2 fixes what belongs inside each activation record. Putting the two together, we can trace the control stack through the **entire** run of the program, one push or pop at a time – 36 events in all, walked in preorder for the calls and postorder for the returns (Section 2.3). Each diagram below shows the complete control stack, top of stack first, down to `main` and the global array, immediately after the labelled push or pop, with every live frame broken down into its own local variables, its call label, and its parameters – the same fields introduced in Figure 3.1.



4. Call q(1,9) <table><tr><td>integer i</td></tr><tr><td>q(1,9)</td></tr><tr><td>integer m, n</td></tr><tr><td>main</td></tr><tr><td>integer a[11] (global)</td></tr></table>	integer i	q(1,9)	integer m, n	main	integer a[11] (global)	5. Call p(1,9) <table><tr><td>p(1,9)</td></tr><tr><td>integer m, n</td></tr><tr><td>integer i</td></tr><tr><td>q(1,9)</td></tr><tr><td>integer m, n</td></tr><tr><td>main</td></tr><tr><td>integer a[11] (global)</td></tr></table>	p(1,9)	integer m, n	integer i	q(1,9)	integer m, n	main	integer a[11] (global)	6. Return p(1,9) <table><tr><td>integer i</td></tr><tr><td>q(1,9)</td></tr><tr><td>integer m, n</td></tr><tr><td>main</td></tr><tr><td>integer a[11] (global)</td></tr></table>	integer i	q(1,9)	integer m, n	main	integer a[11] (global)																					
integer i																																								
q(1,9)																																								
integer m, n																																								
main																																								
integer a[11] (global)																																								
p(1,9)																																								
integer m, n																																								
integer i																																								
q(1,9)																																								
integer m, n																																								
main																																								
integer a[11] (global)																																								
integer i																																								
q(1,9)																																								
integer m, n																																								
main																																								
integer a[11] (global)																																								
7. Call q(1,3) <table><tr><td>integer i</td></tr><tr><td>q(1,3)</td></tr><tr><td>integer m, n</td></tr><tr><td>integer i</td></tr><tr><td>q(1,9)</td></tr><tr><td>integer m, n</td></tr><tr><td>main</td></tr><tr><td>integer a[11] (global)</td></tr></table>	integer i	q(1,3)	integer m, n	integer i	q(1,9)	integer m, n	main	integer a[11] (global)	8. Call p(1,3) <table><tr><td>p(1,3)</td></tr><tr><td>integer m, n</td></tr><tr><td>integer i</td></tr><tr><td>q(1,3)</td></tr><tr><td>integer m, n</td></tr><tr><td>integer i</td></tr><tr><td>q(1,9)</td></tr><tr><td>integer m, n</td></tr><tr><td>main</td></tr><tr><td>integer a[11] (global)</td></tr></table>	p(1,3)	integer m, n	integer i	q(1,3)	integer m, n	integer i	q(1,9)	integer m, n	main	integer a[11] (global)	9. Return p(1,3) <table><tr><td>integer i</td></tr><tr><td>q(1,3)</td></tr><tr><td>integer m, n</td></tr><tr><td>integer i</td></tr><tr><td>q(1,9)</td></tr><tr><td>integer m, n</td></tr><tr><td>main</td></tr><tr><td>integer a[11] (global)</td></tr></table>	integer i	q(1,3)	integer m, n	integer i	q(1,9)	integer m, n	main	integer a[11] (global)												
integer i																																								
q(1,3)																																								
integer m, n																																								
integer i																																								
q(1,9)																																								
integer m, n																																								
main																																								
integer a[11] (global)																																								
p(1,3)																																								
integer m, n																																								
integer i																																								
q(1,3)																																								
integer m, n																																								
integer i																																								
q(1,9)																																								
integer m, n																																								
main																																								
integer a[11] (global)																																								
integer i																																								
q(1,3)																																								
integer m, n																																								
integer i																																								
q(1,9)																																								
integer m, n																																								
main																																								
integer a[11] (global)																																								
10. Call q(1,0) <table><tr><td>integer i</td></tr><tr><td>q(1,0)</td></tr><tr><td>integer m, n</td></tr><tr><td>integer i</td></tr><tr><td>q(1,3)</td></tr><tr><td>integer m, n</td></tr><tr><td>integer i</td></tr><tr><td>q(1,9)</td></tr><tr><td>integer m, n</td></tr><tr><td>main</td></tr><tr><td>integer a[11] (global)</td></tr></table>	integer i	q(1,0)	integer m, n	integer i	q(1,3)	integer m, n	integer i	q(1,9)	integer m, n	main	integer a[11] (global)	11. Return q(1,0) <table><tr><td>integer i</td></tr><tr><td>q(1,3)</td></tr><tr><td>integer m, n</td></tr><tr><td>integer i</td></tr><tr><td>q(1,9)</td></tr><tr><td>integer m, n</td></tr><tr><td>main</td></tr><tr><td>integer a[11] (global)</td></tr></table>	integer i	q(1,3)	integer m, n	integer i	q(1,9)	integer m, n	main	integer a[11] (global)	12. Call q(2,3) <table><tr><td>integer i</td></tr><tr><td>q(2,3)</td></tr><tr><td>integer m, n</td></tr><tr><td>integer i</td></tr><tr><td>q(1,3)</td></tr><tr><td>integer m, n</td></tr><tr><td>integer i</td></tr><tr><td>q(1,9)</td></tr><tr><td>integer m, n</td></tr><tr><td>main</td></tr><tr><td>integer a[11] (global)</td></tr></table>	integer i	q(2,3)	integer m, n	integer i	q(1,3)	integer m, n	integer i	q(1,9)	integer m, n	main	integer a[11] (global)								
integer i																																								
q(1,0)																																								
integer m, n																																								
integer i																																								
q(1,3)																																								
integer m, n																																								
integer i																																								
q(1,9)																																								
integer m, n																																								
main																																								
integer a[11] (global)																																								
integer i																																								
q(1,3)																																								
integer m, n																																								
integer i																																								
q(1,9)																																								
integer m, n																																								
main																																								
integer a[11] (global)																																								
integer i																																								
q(2,3)																																								
integer m, n																																								
integer i																																								
q(1,3)																																								
integer m, n																																								
integer i																																								
q(1,9)																																								
integer m, n																																								
main																																								
integer a[11] (global)																																								
13. Call p(2,3) <table><tr><td>p(2,3)</td></tr><tr><td>integer m, n</td></tr><tr><td>integer i</td></tr><tr><td>q(2,3)</td></tr><tr><td>integer m, n</td></tr><tr><td>integer i</td></tr><tr><td>q(1,3)</td></tr><tr><td>integer m, n</td></tr><tr><td>integer i</td></tr><tr><td>q(1,9)</td></tr><tr><td>integer m, n</td></tr><tr><td>main</td></tr><tr><td>integer a[11] (global)</td></tr></table>	p(2,3)	integer m, n	integer i	q(2,3)	integer m, n	integer i	q(1,3)	integer m, n	integer i	q(1,9)	integer m, n	main	integer a[11] (global)	14. Return p(2,3) <table><tr><td>integer i</td></tr><tr><td>q(2,3)</td></tr><tr><td>integer m, n</td></tr><tr><td>integer i</td></tr><tr><td>q(1,3)</td></tr><tr><td>integer m, n</td></tr><tr><td>integer i</td></tr><tr><td>q(1,9)</td></tr><tr><td>integer m, n</td></tr><tr><td>main</td></tr><tr><td>integer a[11] (global)</td></tr></table>	integer i	q(2,3)	integer m, n	integer i	q(1,3)	integer m, n	integer i	q(1,9)	integer m, n	main	integer a[11] (global)	15. Call q(2,1) <table><tr><td>integer i</td></tr><tr><td>q(2,1)</td></tr><tr><td>integer m, n</td></tr><tr><td>integer i</td></tr><tr><td>q(2,3)</td></tr><tr><td>integer m, n</td></tr><tr><td>integer i</td></tr><tr><td>q(1,3)</td></tr><tr><td>integer m, n</td></tr><tr><td>integer i</td></tr><tr><td>q(1,9)</td></tr><tr><td>integer m, n</td></tr><tr><td>main</td></tr><tr><td>integer a[11] (global)</td></tr></table>	integer i	q(2,1)	integer m, n	integer i	q(2,3)	integer m, n	integer i	q(1,3)	integer m, n	integer i	q(1,9)	integer m, n	main	integer a[11] (global)
p(2,3)																																								
integer m, n																																								
integer i																																								
q(2,3)																																								
integer m, n																																								
integer i																																								
q(1,3)																																								
integer m, n																																								
integer i																																								
q(1,9)																																								
integer m, n																																								
main																																								
integer a[11] (global)																																								
integer i																																								
q(2,3)																																								
integer m, n																																								
integer i																																								
q(1,3)																																								
integer m, n																																								
integer i																																								
q(1,9)																																								
integer m, n																																								
main																																								
integer a[11] (global)																																								
integer i																																								
q(2,1)																																								
integer m, n																																								
integer i																																								
q(2,3)																																								
integer m, n																																								
integer i																																								
q(1,3)																																								
integer m, n																																								
integer i																																								
q(1,9)																																								
integer m, n																																								
main																																								
integer a[11] (global)																																								

16. Return q(2,1)

integer i
q(2,3)
integer m, n
integer i
q(1,3)
integer m, n
integer i
q(1,9)
integer m, n
main
integer a[11] (global)

17. Call q(3,3)

integer i
q(3,3)
integer m, n
integer i
q(2,3)
integer m, n
integer i
q(1,3)
integer m, n
integer i
q(1,9)
integer m, n
main
integer a[11] (global)

18. Return q(3,3)

integer i
q(2,3)
integer m, n
integer i
q(1,3)
integer m, n
integer i
q(1,9)
integer m, n
main
integer a[11] (global)

19. Return q(2,3)

integer i
q(1,3)
integer m, n
integer i
q(1,9)
integer m, n
main
integer a[11] (global)

20. Return q(1,3)

integer i
q(1,9)
integer m, n
main
integer a[11] (global)

21. Call q(5,9)

integer i
q(5,9)
integer m, n
integer i
q(1,9)
integer m, n
main
integer a[11] (global)

22. Call p(5,9)

p(5,9)
integer m, n
integer i
q(5,9)
integer m, n
integer i
q(1,9)
integer m, n
main
integer a[11] (global)

23. Return p(5,9)

integer i
q(5,9)
integer m, n
integer i
q(1,9)
integer m, n
main
integer a[11] (global)

24. Call q(5,5)

integer i
q(5,5)
integer m, n
integer i
q(5,9)
integer m, n
integer i
q(1,9)
integer m, n
main
integer a[11] (global)

25. Return q(5,5)

integer i
q(5,9)
integer m, n
integer i
q(1,9)
integer m, n
main
integer a[11] (global)

26. Call q(7,9)

integer i
q(7,9)
integer m, n
integer i
q(5,9)
integer m, n
integer i
q(1,9)
integer m, n
main
integer a[11] (global)

27. Call p(7,9)

p(7,9)
integer m, n
integer i
q(7,9)
integer m, n
integer i
q(5,9)
integer m, n
integer i
q(1,9)
integer m, n
main
integer a[11] (global)

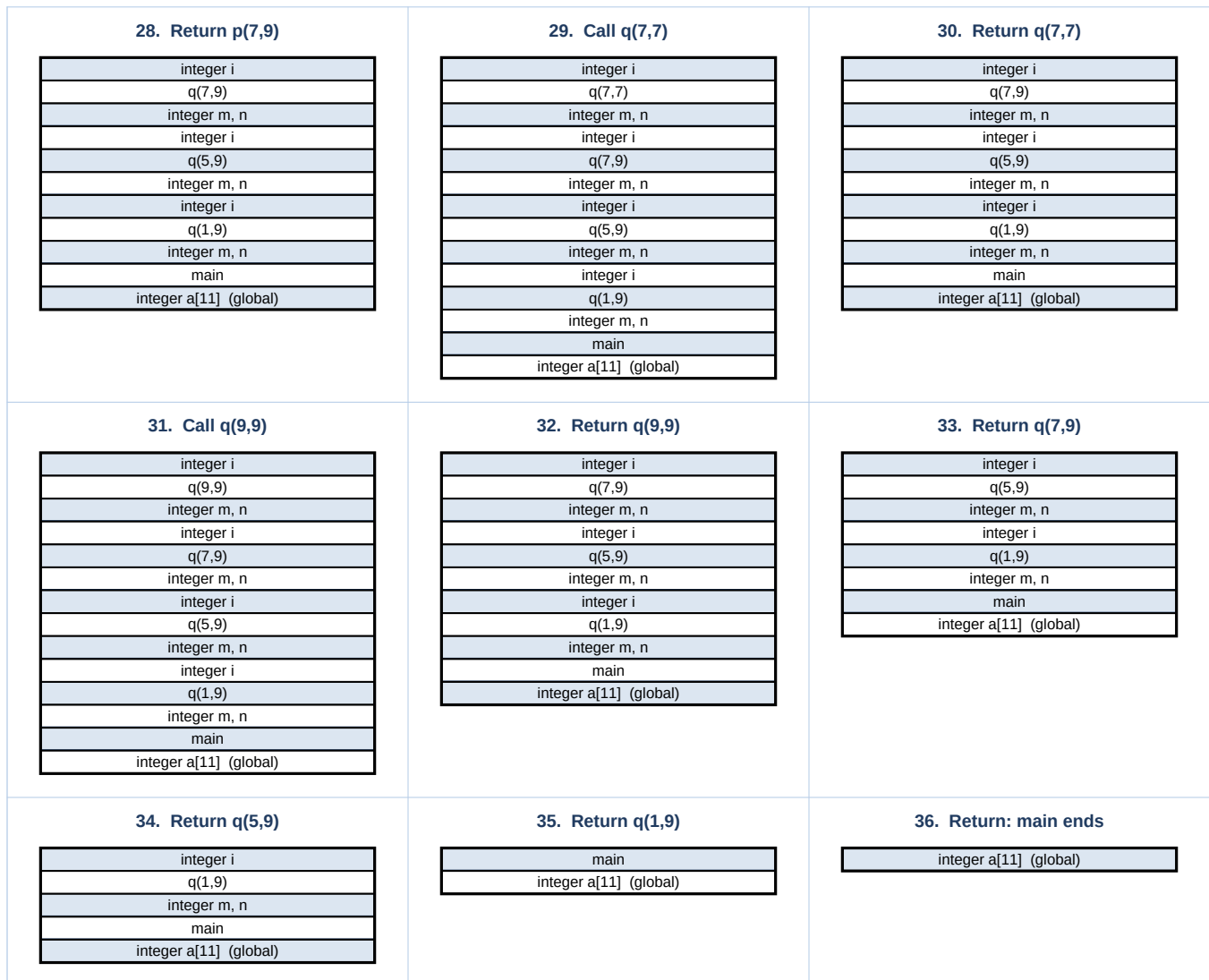


Figure 3.4 – All 36 steps of the control-stack trace for the quicksort run of Figure 2.1, redrawn from the original step-by-step slide deck. Read left-to-right, top-to-bottom; each panel shows the complete stack immediately after that step's push or pop.

Theory Questions

Q. What is the difference between the “control stack” and an “activation record”?

A. The control stack is the run-time stack itself – the whole structure that manages procedure calls and returns. An activation record (or frame) is a single entry on that stack: the block of storage dedicated to one live activation.

Q. Where does the record for main sit on the control stack, and why?

A. At the very bottom (just above the global data area, if any) – because main is the root of the activation tree and, being the first procedure to start, it is the last one to finish; nothing sits below it once the program is running.

Q. Why can space once used by an activation of r be safely reused by a later, unrelated activation of q?

A. Because activations nest in time: once r has returned, it is no longer live, and nothing later in the program can ever need its data again, so the space it occupied is free to be reused by whatever is pushed next.

Q. Why is it normal for a recursive procedure to have many of its own activation records on the stack at the same time?

A. Each recursive call is a distinct activation (with its own parameters/locals) that stays live until it returns; since deeper recursive calls must finish before the call that made them can finish, several still-open calls along the current path of the recursion pile up on the stack at once, one record per open call.

Practice Problem

Source: CSE420 Final Exam, Fall 2025, Set A – Question 7

"Draw an activation tree for these function call sequences and draw the state of the stack at the flow of execution at node f(3)."

```
int b(int n) {  
    return n;  
}  
int f(int n) {  
    if (n == 0) return 0;  
    return b(n) + f(n-1);  
}  
int main() {  
    int x;  
    x = f(5);  
}
```

Solution. Assuming the standard left-to-right evaluation of $b(n) + f(n-1)$ (the same left-to-right assumption used for `quicksort(m, i-1)` then `quicksort(i+1, n)` in Section 2.4), $b(n)$ is always called and returns before $f(n-1)$ is entered. Tracing the calls: main calls $f(5)$; since $5 \neq 0$, $f(5)$ calls $b(5)$ (which returns immediately) and then $f(4)$; $f(4)$ calls $b(4)$ then $f(3)$; $f(3)$ calls $b(3)$ then $f(2)$; $f(2)$ calls $b(2)$ then $f(1)$; $f(1)$ calls $b(1)$ then $f(0)$; $f(0)$ hits the base case ($n=0$) and returns 0 directly, without calling b or f again.

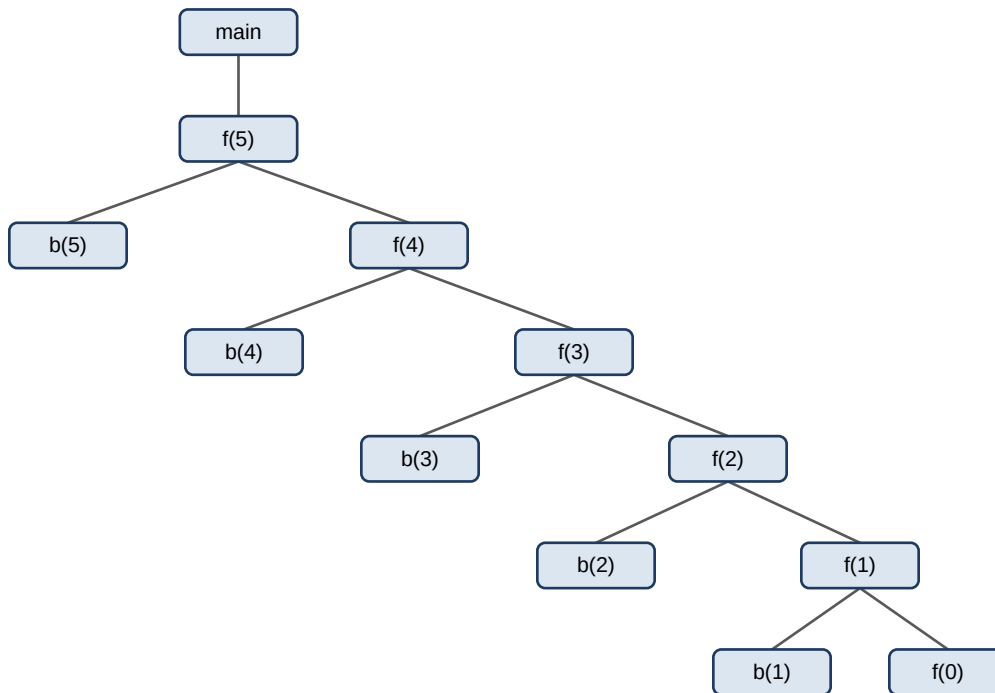


Figure 3.6 – Activation tree for main / f / b.

State of the stack at node f(3). By the live-activation rule (Section 2.3, property 3), the activations open when control is inside f(3) are exactly f(3) and its ancestors: the path $\text{main} \rightarrow \text{f}(5) \rightarrow \text{f}(4) \rightarrow \text{f}(3)$. Note that b(5) and b(4) are *not* ancestors of f(3) – each b(k) is called and returns before f(k-1) is ever entered, so by the time f(3) is reached, both b(5) and b(4) have already been popped off the stack.

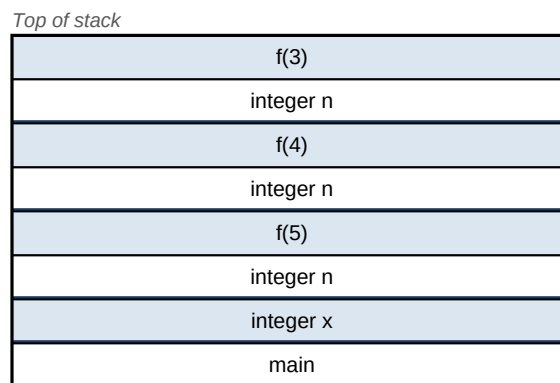


Figure 3.7 – Control stack when execution is inside f(3).

Complete stack simulation. The single snapshot above answers the exam question, but the same activation tree (Figure 3.6) gives us the entire run of the program, one push or pop at a time – walked in preorder for the calls and postorder for the returns, exactly as in Section 3.5. Every live frame below is broken down into its parameters and call label (main additionally has its own local variable x); there is no global data in this program, so the stack empties completely once main returns.

1. Start: frame for main <table><tr><td>integer x</td></tr><tr><td>main</td></tr></table>	integer x	main	2. Call f(5) <table><tr><td>f(5)</td></tr><tr><td>integer n</td></tr><tr><td>integer x</td></tr><tr><td>main</td></tr></table>	f(5)	integer n	integer x	main	3. Call b(5) <table><tr><td>b(5)</td></tr><tr><td>integer n</td></tr><tr><td>f(5)</td></tr><tr><td>integer n</td></tr><tr><td>integer x</td></tr><tr><td>main</td></tr></table>	b(5)	integer n	f(5)	integer n	integer x	main																								
integer x																																						
main																																						
f(5)																																						
integer n																																						
integer x																																						
main																																						
b(5)																																						
integer n																																						
f(5)																																						
integer n																																						
integer x																																						
main																																						
4. Return b(5) <table><tr><td>f(5)</td></tr><tr><td>integer n</td></tr><tr><td>integer x</td></tr><tr><td>main</td></tr></table>	f(5)	integer n	integer x	main	5. Call f(4) <table><tr><td>f(4)</td></tr><tr><td>integer n</td></tr><tr><td>f(5)</td></tr><tr><td>integer n</td></tr><tr><td>integer x</td></tr><tr><td>main</td></tr></table>	f(4)	integer n	f(5)	integer n	integer x	main	6. Call b(4) <table><tr><td>b(4)</td></tr><tr><td>integer n</td></tr><tr><td>f(4)</td></tr><tr><td>integer n</td></tr><tr><td>f(5)</td></tr><tr><td>integer n</td></tr><tr><td>integer x</td></tr><tr><td>main</td></tr></table>	b(4)	integer n	f(4)	integer n	f(5)	integer n	integer x	main																		
f(5)																																						
integer n																																						
integer x																																						
main																																						
f(4)																																						
integer n																																						
f(5)																																						
integer n																																						
integer x																																						
main																																						
b(4)																																						
integer n																																						
f(4)																																						
integer n																																						
f(5)																																						
integer n																																						
integer x																																						
main																																						
7. Return b(4) <table><tr><td>f(4)</td></tr><tr><td>integer n</td></tr><tr><td>f(5)</td></tr><tr><td>integer n</td></tr><tr><td>integer x</td></tr><tr><td>main</td></tr></table>	f(4)	integer n	f(5)	integer n	integer x	main	8. Call f(3) <table><tr><td>f(3)</td></tr><tr><td>integer n</td></tr><tr><td>f(4)</td></tr><tr><td>integer n</td></tr><tr><td>f(5)</td></tr><tr><td>integer n</td></tr><tr><td>integer x</td></tr><tr><td>main</td></tr></table>	f(3)	integer n	f(4)	integer n	f(5)	integer n	integer x	main	9. Call b(3) <table><tr><td>b(3)</td></tr><tr><td>integer n</td></tr><tr><td>f(3)</td></tr><tr><td>integer n</td></tr><tr><td>f(4)</td></tr><tr><td>integer n</td></tr><tr><td>f(5)</td></tr><tr><td>integer n</td></tr><tr><td>integer x</td></tr><tr><td>main</td></tr></table>	b(3)	integer n	f(3)	integer n	f(4)	integer n	f(5)	integer n	integer x	main												
f(4)																																						
integer n																																						
f(5)																																						
integer n																																						
integer x																																						
main																																						
f(3)																																						
integer n																																						
f(4)																																						
integer n																																						
f(5)																																						
integer n																																						
integer x																																						
main																																						
b(3)																																						
integer n																																						
f(3)																																						
integer n																																						
f(4)																																						
integer n																																						
f(5)																																						
integer n																																						
integer x																																						
main																																						
10. Return b(3) <table><tr><td>f(3)</td></tr><tr><td>integer n</td></tr><tr><td>f(4)</td></tr><tr><td>integer n</td></tr><tr><td>f(5)</td></tr><tr><td>integer n</td></tr><tr><td>integer x</td></tr><tr><td>main</td></tr></table>	f(3)	integer n	f(4)	integer n	f(5)	integer n	integer x	main	11. Call f(2) <table><tr><td>f(2)</td></tr><tr><td>integer n</td></tr><tr><td>f(3)</td></tr><tr><td>integer n</td></tr><tr><td>f(4)</td></tr><tr><td>integer n</td></tr><tr><td>f(5)</td></tr><tr><td>integer n</td></tr><tr><td>integer x</td></tr><tr><td>main</td></tr></table>	f(2)	integer n	f(3)	integer n	f(4)	integer n	f(5)	integer n	integer x	main	12. Call b(2) <table><tr><td>b(2)</td></tr><tr><td>integer n</td></tr><tr><td>f(2)</td></tr><tr><td>integer n</td></tr><tr><td>f(3)</td></tr><tr><td>integer n</td></tr><tr><td>f(4)</td></tr><tr><td>integer n</td></tr><tr><td>f(5)</td></tr><tr><td>integer n</td></tr><tr><td>integer x</td></tr><tr><td>main</td></tr></table>	b(2)	integer n	f(2)	integer n	f(3)	integer n	f(4)	integer n	f(5)	integer n	integer x	main						
f(3)																																						
integer n																																						
f(4)																																						
integer n																																						
f(5)																																						
integer n																																						
integer x																																						
main																																						
f(2)																																						
integer n																																						
f(3)																																						
integer n																																						
f(4)																																						
integer n																																						
f(5)																																						
integer n																																						
integer x																																						
main																																						
b(2)																																						
integer n																																						
f(2)																																						
integer n																																						
f(3)																																						
integer n																																						
f(4)																																						
integer n																																						
f(5)																																						
integer n																																						
integer x																																						
main																																						
13. Return b(2) <table><tr><td>f(2)</td></tr><tr><td>integer n</td></tr><tr><td>f(3)</td></tr><tr><td>integer n</td></tr><tr><td>f(4)</td></tr><tr><td>integer n</td></tr><tr><td>f(5)</td></tr><tr><td>integer n</td></tr><tr><td>integer x</td></tr><tr><td>main</td></tr></table>	f(2)	integer n	f(3)	integer n	f(4)	integer n	f(5)	integer n	integer x	main	14. Call f(1) <table><tr><td>f(1)</td></tr><tr><td>integer n</td></tr><tr><td>f(2)</td></tr><tr><td>integer n</td></tr><tr><td>f(3)</td></tr><tr><td>integer n</td></tr><tr><td>f(4)</td></tr><tr><td>integer n</td></tr><tr><td>f(5)</td></tr><tr><td>integer n</td></tr><tr><td>integer x</td></tr><tr><td>main</td></tr></table>	f(1)	integer n	f(2)	integer n	f(3)	integer n	f(4)	integer n	f(5)	integer n	integer x	main	15. Call b(1) <table><tr><td>b(1)</td></tr><tr><td>integer n</td></tr><tr><td>f(1)</td></tr><tr><td>integer n</td></tr><tr><td>f(2)</td></tr><tr><td>integer n</td></tr><tr><td>f(3)</td></tr><tr><td>integer n</td></tr><tr><td>f(4)</td></tr><tr><td>integer n</td></tr><tr><td>f(5)</td></tr><tr><td>integer n</td></tr><tr><td>integer x</td></tr><tr><td>main</td></tr></table>	b(1)	integer n	f(1)	integer n	f(2)	integer n	f(3)	integer n	f(4)	integer n	f(5)	integer n	integer x	main
f(2)																																						
integer n																																						
f(3)																																						
integer n																																						
f(4)																																						
integer n																																						
f(5)																																						
integer n																																						
integer x																																						
main																																						
f(1)																																						
integer n																																						
f(2)																																						
integer n																																						
f(3)																																						
integer n																																						
f(4)																																						
integer n																																						
f(5)																																						
integer n																																						
integer x																																						
main																																						
b(1)																																						
integer n																																						
f(1)																																						
integer n																																						
f(2)																																						
integer n																																						
f(3)																																						
integer n																																						
f(4)																																						
integer n																																						
f(5)																																						
integer n																																						
integer x																																						
main																																						

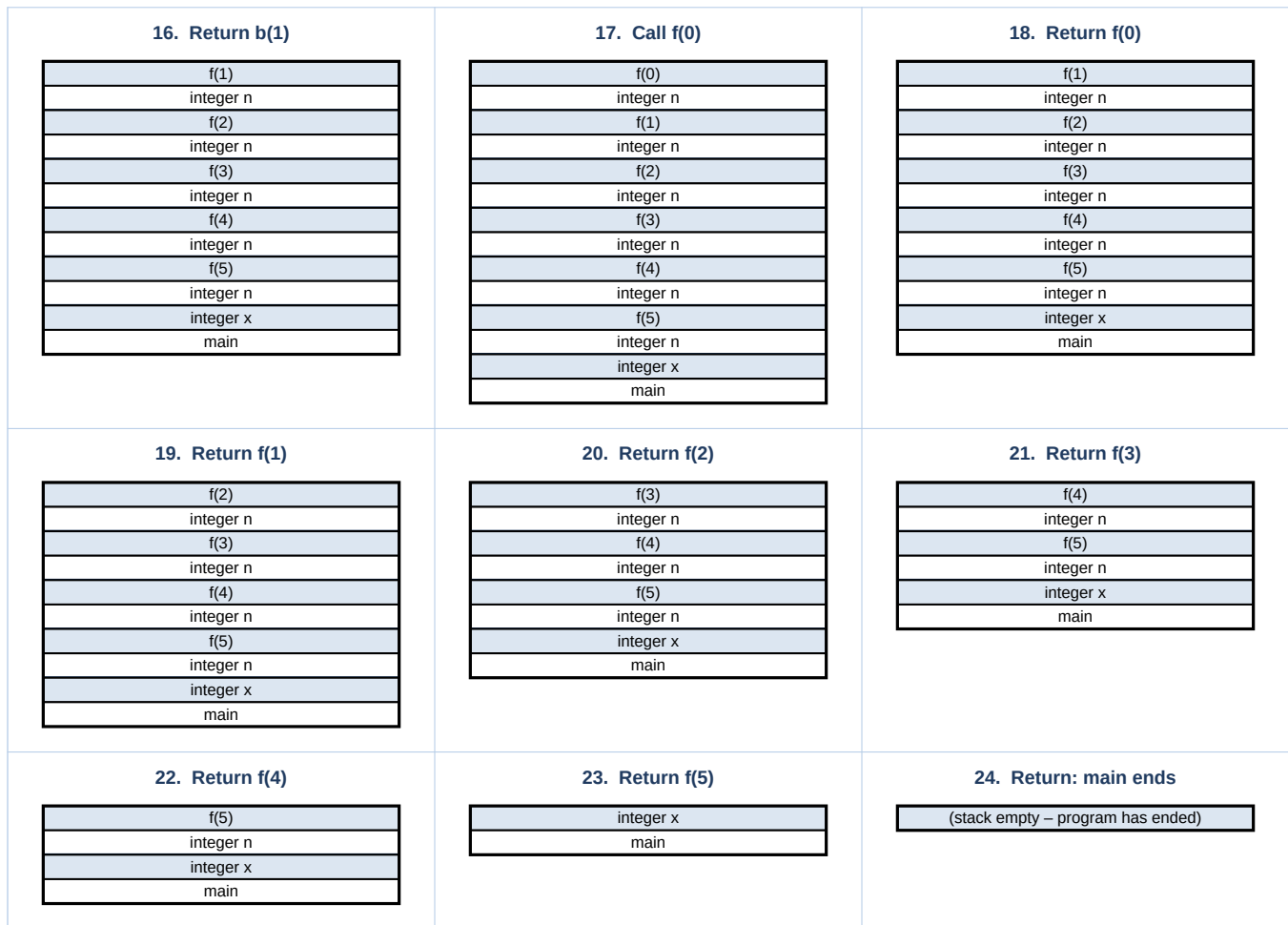


Figure 3.8 – All 24 steps of the control-stack trace for the main / f / b program of Figure 3.6. Steps 8 and 10 both match Figure 3.7, confirming that the stack has the same shape at every point where control is directly inside f(3) (before and after its call to b(3)).

4. Calling Sequences

4.1 Calling and Return Sequences

Procedure calls are implemented with a **calling sequence**: code that allocates an activation record on the stack and fills in its fields. A matching **return sequence** restores the machine's state so the caller can continue after the call. Calling sequences and activation-record layout can differ a great deal, even between implementations of the same language, because there is no fixed division of labour between the caller and the callee – the source language, the target machine, and the operating system all push the design one way or another.

If a procedure is called from n different call sites, the caller's portion of the calling sequence is generated n times – but the callee's portion is generated only once, since it lives inside the procedure itself. So, wherever possible, it pays to put as much of the calling sequence into the **callee** as it can be trusted to know – though, as the principles below show, the callee cannot know everything about every call site.

4.2 Design Principles

1. **Values shared between caller and callee go at the beginning of the callee's record**, right next to the caller's record. This lets the caller compute and place the actual parameters on top of its own record without

needing to know the callee's whole layout, and it supports procedures that take a variable number or type of arguments (e.g. C's `printf`). The callee always knows where its return value goes, relative to its own record, and can find however many arguments follow it.

2. **Fixed-length items go in the middle** – typically the control link, access link, and saved machine-status fields. Saving and restoring the same components for every call lets the same code do it every time, and standardizing this layout also makes life easier for tools such as debuggers that need to walk the stack.

3. **Items whose size is not known early enough go at the end.** Most local variables have a fixed, compile-time-known size, but some – a dynamically-sized local array whose length depends on a parameter, for example – do not; likewise, the space needed for temporaries depends on how many the code generator manages to keep in registers, which may not be settled when intermediate code is first produced.

4. **Locate the top-of-stack pointer judiciously.** A common choice is to have it point to the end of the fixed-length fields in the record, so fixed data can be reached with fixed, compile-time-known offsets.

4.3 The `top_sp` Register

A register, commonly named `top_sp`, points to the end of the machine-status field of the current (topmost) activation record. This is the one boundary inside the frame whose position is always known at compile time – everything below it, in stack-growth order, is fixed-length, and everything above it (locals, temporaries) may be variable-length. So the intermediate-code generator can reach every fixed field with a constant offset from `top_sp`. Variable-length fields sit logically “above” `top_sp`; their exact offsets must be computed at run time, but they too can be reached from `top_sp` using a positive offset.

Figure 4.1 makes this concrete by showing two consecutive activation records on the stack – the caller's, already established, and the callee's, currently being built – and marking exactly where each party's job begins and ends.

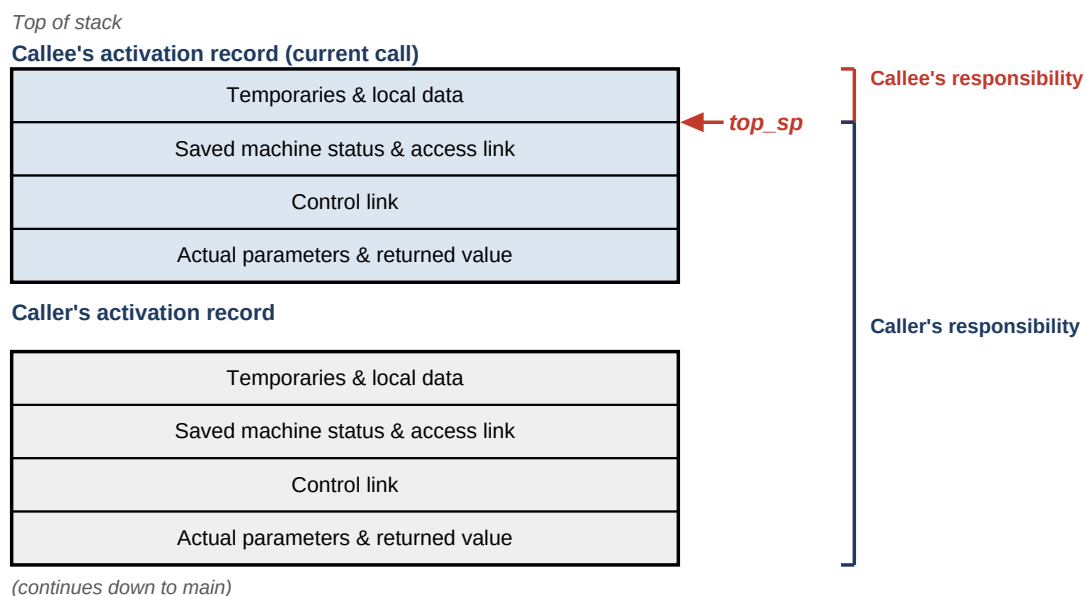


Figure 4.1 – Division of tasks between caller and callee. `top_sp` marks the boundary, inside the callee's own record, that the caller sets before transferring control: everything below that boundary – the callee's parameters, control link, and saved status, plus the whole of the caller's own record beneath it – is the **caller's responsibility**. Only the callee's own temporaries and local data, above `top_sp`, are the **callee's responsibility** to fill in once control reaches it.

4.4 The Calling Sequence, Step by Step

Calling sequence (as control passes from caller to callee):

1. The caller evaluates the actual parameters.
2. The caller stores a return address and the old value of `top_sp` into the callee's activation record, then advances `top_sp` to the position shown in Figure 4.1 – i.e. past the caller's own locals/temporaries and past the callee's parameter and status fields.
3. The callee saves the register values and other status information it will need to restore later.
4. The callee initializes its own local data and begins executing.

Return sequence (as control passes back from callee to caller):

1. The callee places the return value next to the parameters (Figure 3.1).
2. Using the saved machine-status field, the callee restores `top_sp` and any other saved registers, then branches to the return address the caller had stored.
3. Although `top_sp` has now been decremented, the caller still knows – relative to the (restored) value of `top_sp` – exactly where the return value is, so it simply uses it.

This scheme lets the number of arguments vary from call to call, as with C's `printf`: at compile time, the caller's target code knows exactly how many/what type of arguments it is passing for this call, so it knows the size of the parameter area. The callee's code, however, must stay ready for any call, so it waits until it is actually invoked and then examines the parameter field it has been handed – which it can find because parameter information is always placed right next to the status field.

4.5 Variable-Length Data on the Stack

Not every local object has a size the compiler can pin down in advance – a dynamically-sized local array, for instance, may depend on the value of a parameter. Whenever such an object is local to a procedure and becomes inaccessible once that procedure returns, it is still worth allocating it on the **stack** rather than the heap, because doing so avoids the expense of garbage collecting it later. The specific bookkeeping technique for this case (7.2.4 in the reference textbook) is beyond the excerpt used for this note.

Theory Questions

Q. Why is the callee's portion of a calling sequence generated only once, while the caller's portion is generated once per call site?

A. The callee's code lives inside the procedure body itself, so it exists exactly once no matter how many places call it; the caller's portion is emitted inline at every call site, so it is duplicated n times for n call sites. This is exactly why calling-sequence design tries to push as much responsibility as possible onto the callee.

Q. Why must the caller, not the callee, evaluate the actual parameters and set `top_sp` before the call?

A. Only the caller knows, at compile time, what values it wants to pass and how many/what type of arguments this particular call has; the callee's code must stay generic enough to serve every call site, so it cannot know in advance what a specific caller is about to send it.

Practice Problems

Source: CSE420 Final Exam, Fall 2025, Set A – Question 2

"Explain what the register stack pointer (`top_sp`) represents in a run-time stack. Why is it placed at the end of the fixed-length fields in an activation record?"

Solution. `top_sp` is a register that always points to the end of the machine-status field – equivalently, the boundary between the fixed-length and variable-length parts – of whichever activation record currently sits on top of the

control stack. It is effectively the base address the intermediate-code generator uses to reach the current activation's fixed-length fields (parameters, returned-value slot, control link, access link, saved machine status) using constant, compile-time-known offsets.

It is placed at the end of the fixed-length fields, rather than at the very top of the whole frame, because that boundary is the one point inside the record whose position can be pinned down at compile time no matter what happens above it. Local data and temporaries – the fields that come next – may have sizes that are not known until well into compilation, or even until run time for dynamically-sized locals, so they are pushed “above” `top_sp`. Anchoring `top_sp` right after the fixed part means every fixed field keeps the same constant offset regardless of how large the variable part later turns out to be, and it lets the caller set `top_sp` before transferring control, since the caller already knows the size of everything up to that point without needing to know the callee's local-variable layout.

Source: CSE420 Final Exam, Spring 2026, Set A – Question 2

"Why are temporaries and local data kept at the end of the activation record? Which components of a callee's activation record is the callee responsible for and why?"

Solution. *Part 1.* Temporaries and local data go at the end (top) of the record because their sizes are frequently not known early enough to fix an offset for them ahead of time. Most locals do have a compile-time-known size, but some – a dynamically-sized array whose length depends on a parameter, for example – do not, and the space needed for temporaries depends on how many the code-generation phase manages to keep in registers, which may not be settled when intermediate code is first generated. Keeping these two variable-sized fields at the very end means the fixed-length fields below them (parameters, links, saved status) always keep the same offsets, so they can be found reliably even before the size of the locals/temporaries is pinned down.

Part 2. Following the calling sequence of Section 4.4, the callee is responsible for saving the register values and other status information (into its own saved-machine-status field) and for initializing its own local data and temporaries before it begins executing; symmetrically, on return, it is the callee that places the return value and restores `top_sp`/registers before branching back. It handles these fields itself because only the callee's own code knows what registers it will use and what local storage it needs – the caller cannot know these implementation details of a procedure it merely calls, so this part of the calling sequence is generated once, inside the callee, instead of being duplicated (incorrectly) at every call site.

Source: CSE420 Final Exam, Summer 2025, Set A – Question 2

"Draw the content of the activation record of a function in the runtime stack, and explain how parameter passing and value returning between caller and callee happens."

Solution. The general activation record is exactly the one built up in Section 3.2 (Figure 3.1): temporaries and local data at the top (current call's own working storage), saved machine status, access link, and control link in the fixed middle, and the returned value and actual parameters at the bottom of the frame, adjoining the caller.

Parameter passing. Before transferring control, the caller evaluates each actual parameter and places the values in the parameter area at the start of the callee's (about-to-exist) record, right next to the top of its own record; it also stores the return address and the old value of `top_sp` there, then advances `top_sp` past its own data and past the callee's parameter/status fields (Section 4.4). Control now passes to the callee, which reads its parameters using fixed, `top_sp`-relative offsets – exactly as the caller laid them out.

Returning a value. When the callee is ready to return, it writes the return value into the reserved slot right beside the parameters. It then restores `top_sp` and any saved registers from its saved-machine-status field and branches to the return address the caller had stored. Back in the caller, although `top_sp` has now been decremented, the caller still knows exactly where the return value sits relative to the (restored) `top_sp`, so it simply reads it from there.